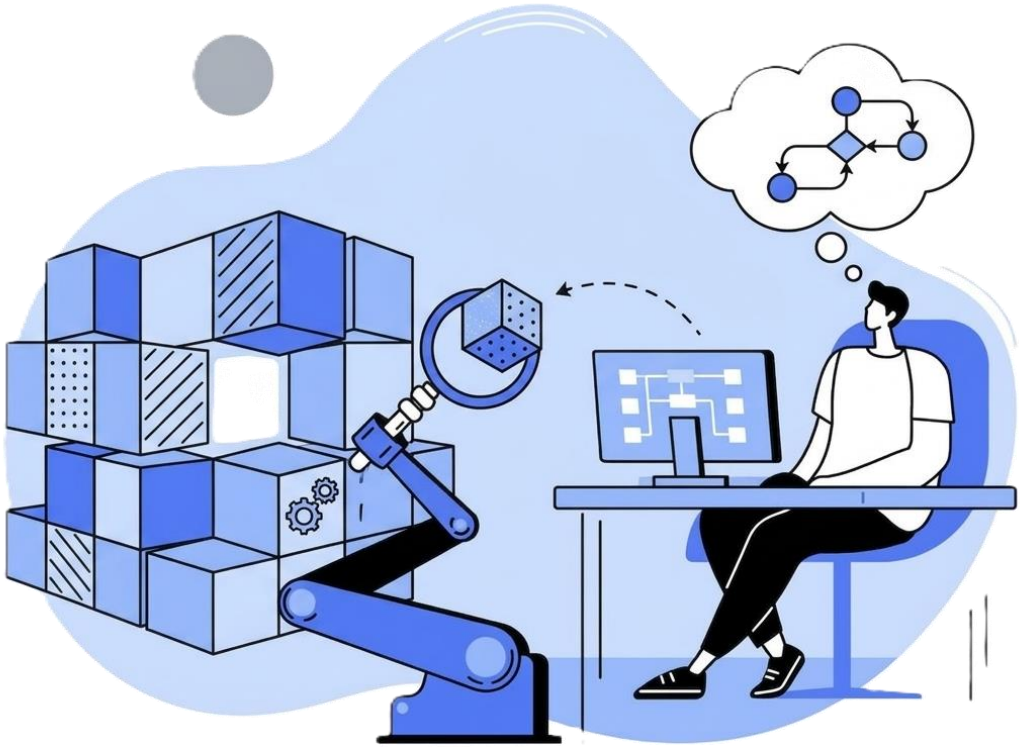
Principes SOLID & Design Patterns

avec Java & Intellij IDE



''Pour une architecture facile a comprendre, maintenir et plus flexible''

Lien des ressources du code : *https://github.com/Hicham-El-Mouloudi/SOLID-Principles-Presentation*

Réalisé par : **Hicham El Mouloudi**

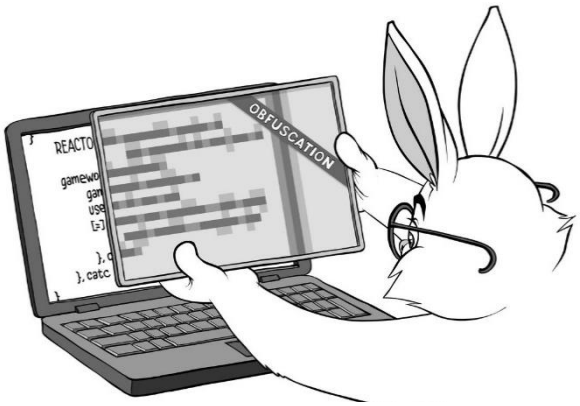
Sommaire

I. Les Principes de la programation orientée objet : 3
1. Encapsulation: 3
2. Abstraction: 3
3. Héritage: 3
4. Polymorphisme: 3
II. C'est quoi La constante en Dévellopement Informatique ? 4
III. Composition & Délégation : 4
IV. Le dilemme entre Composition et Héritage pour la réutilisation de code : 4
1. Les limites de la réutilisation du code par héritage 4
V. Les Principes SOLID : 5
1. Single Responsabilité Principle : 6
2. Open/Closed Principle : 7
3. Liskov Substitution Principle : 7
4. Interface Segregation Principle : 8
5. Dependency Inversion Principle : 9
VI. Conclusion : 10

I. Les Principes de la programation orientée objet :

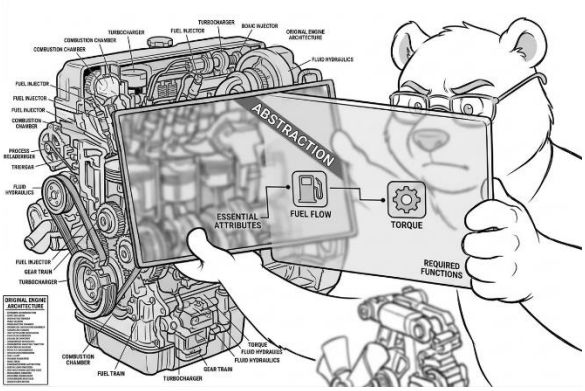
1. Encapsulation:

Regroupement de l'état et du comportement au sein d'une unité logique, restreignant l'accès direct aux composants internes via des modificateurs de portée pour garantir l'intégrité des données.



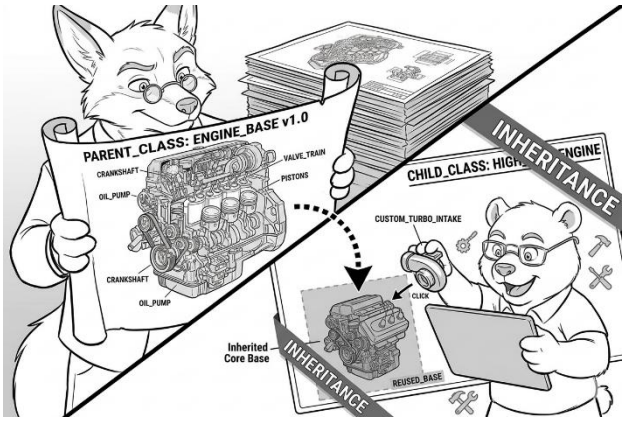
2. Abstraction:

Modélisation des caractéristiques essentielles d'une entité en séparant l'interface publique de l'implémentation technique, permettant de réduire la complexité systémique.



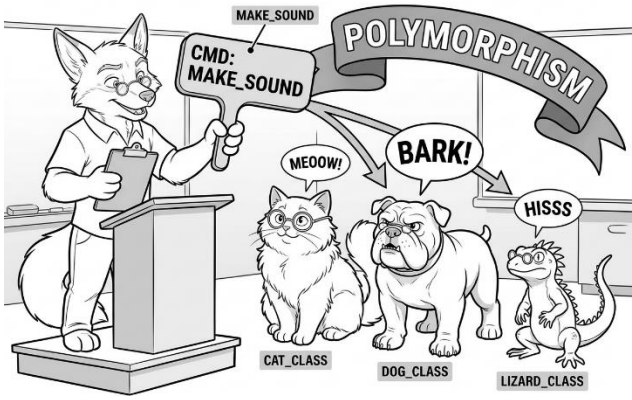
3. Héritage:

Mécanisme de dérivation hiérarchique permettant à une classe de s'approprier les attributs et méthodes d'une classe parente pour favoriser la spécialisation et la réutilisabilité.



4. Polymorphisme:

Faculté d'une interface à représenter plusieurs types d'objets, permettant à une même signature de méthode de déclencher des comportements distincts via le typage dynamique.



II. C’est quoi La constante en Dévellopement Informatique ?

En informatique, un logiciel n'est jamais figé. Les besoins clients évoluent, les technos vieillissent et les bugs surgissent. Si le code est **rigide**, chaque modification devient un risque.

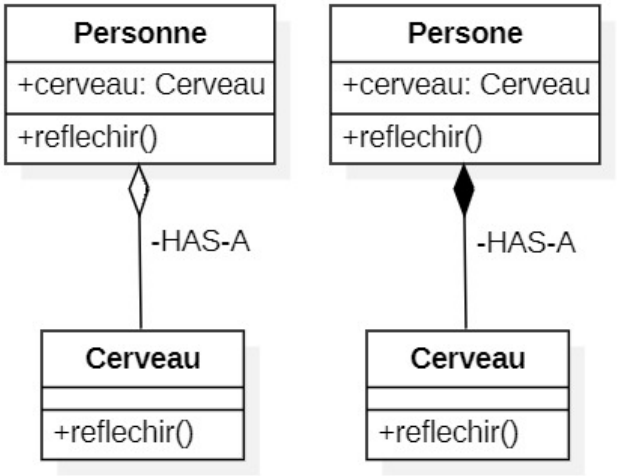
Solution:

- Les principes **SOLID** et les bonnes pratiques transforment cette contrainte en force en rendant le code **souple. On ne développe pas pour que ça "marche" aujourd'hui, on développe pour que ce soit "modifiable" demain.**

III. Composition & Délégation :

Composition = est une approche de conception où l'on assemble des composants petits, indépendants et réutilisables pour créer un système plus vaste et complexe.

Délégation = une classe confie l’exécution d’une tâche à un objet **qu’elle possède**.



Note : En bonnes pratiques, on regroupe deux mécanismes sous le terme générique de "Composition" :

- **Composition "Pure" (Statique) : L'objet contenant instancie ses composants en interne. Les cycles de vie sont liés : si le parent meurt, le composant meurt.**
- **Agrégation (Dynamique) : L'objet reçoit ses composants via l'injection de dépendance (constructeur). Les cycles de vie sont indépendants : le composant survit au parent.**

IV. Le dilemme entre Composition et Héritage pour la réutilisation de code :

Dans la conception logicielle, la réutilisation de code est un pilier fondamental, mais elle souligne un dilemme classique : faut-il utiliser l'**Héritage** ou la **Composition** ?

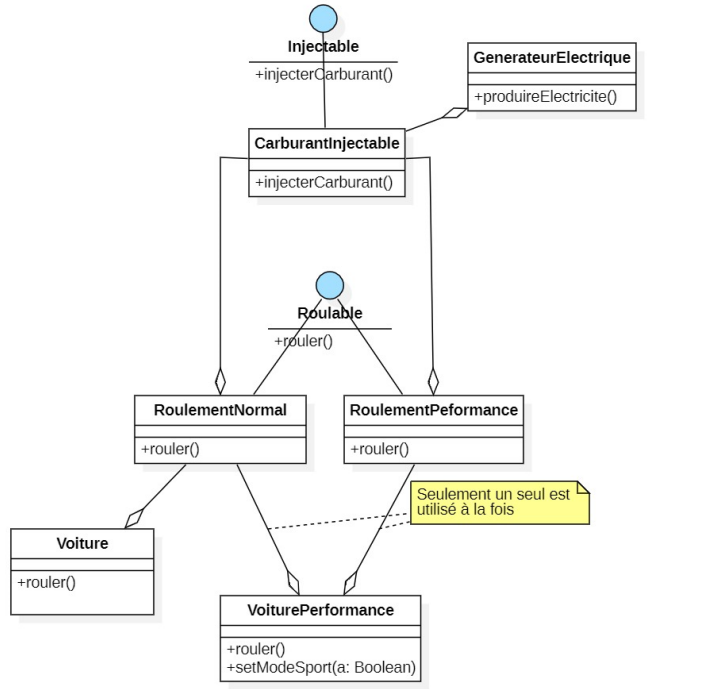
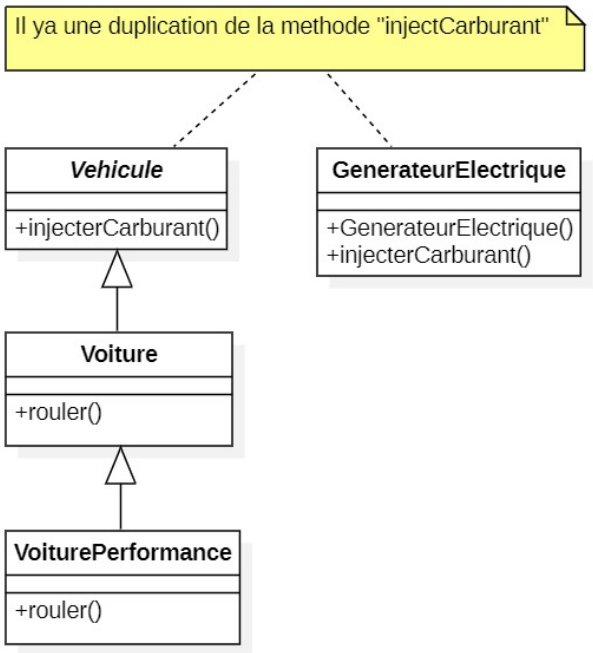
1. Les limites de la réutilisation du code par héritage

- Duplication du code

- Une erreur non intentionnelle peut affecter les sous-classes
- Difficulté à savoir quel comportement possède une sous-classe
- L'implémentation du comportement est statique dès que la sous-classe est instanciée

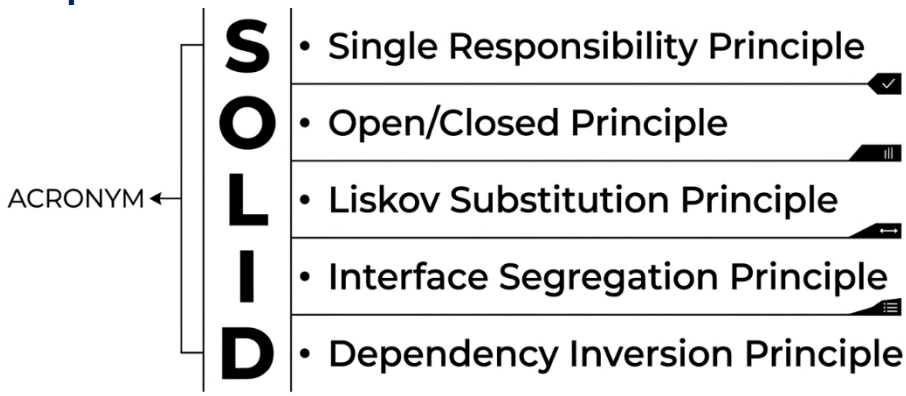
Le couplage de la **composition** et de la **délégation** dépasse ces contraintes. C'est une approche optimale de la réutilisation qui évite les inconvénients structurels imposés par l'héritage.

- **Exemple:** (**Voir les packages :** ***'org.springformation.solid.principles.examples.composition_ou_heritage.(sale/propre)'***)

Approche par Composition&Délégation	Approche par Héritage
	

Note : L'approche par composition et délégation offre une solution plus flexible que celle de l'héritage, surtout pour la réutilisation du code. Cependant, cette flexibilité a un coût : l'augmentation du nombre de classes et d'interfaces, qui peut parfois doubler par rapport à une structure classique par héritage.

V. Les Principes SOLID :



1. Single Responsabilité Principle :

Énoncé : **‘Une classe ne devrait avoir qu'une seule raison de changer’**

- Le Single Responsibility Principle stipule qu'une classe ne doit avoir qu'une seule raison de changer, ce qui signifie qu'elle doit se limiter à une seule et unique fonction pour garantir un code modulaire et facile à maintenir.

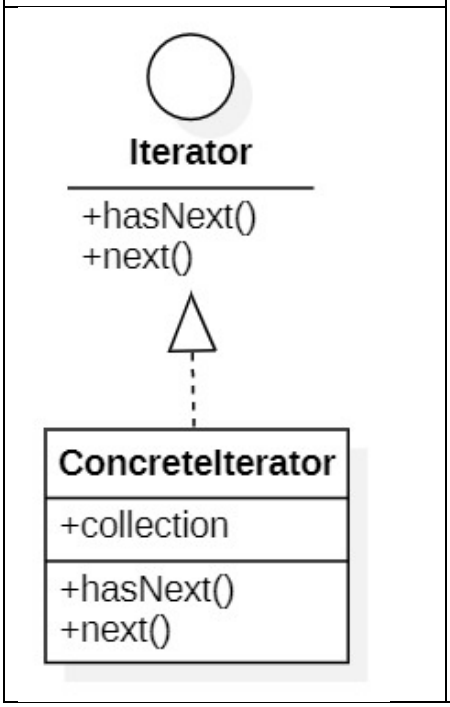
a. La Cohésion : un indicateur de bonne application du SRP :
La **cohésion** mesure le degré d'unité d'une classe : une **forte cohésion** signifie que toutes les méthodes et attributs collaborent vers un **but commun**, prouvant ainsi que le **SRP** est respecté et que la classe possède une mission **unique**, claire et non dispersée.

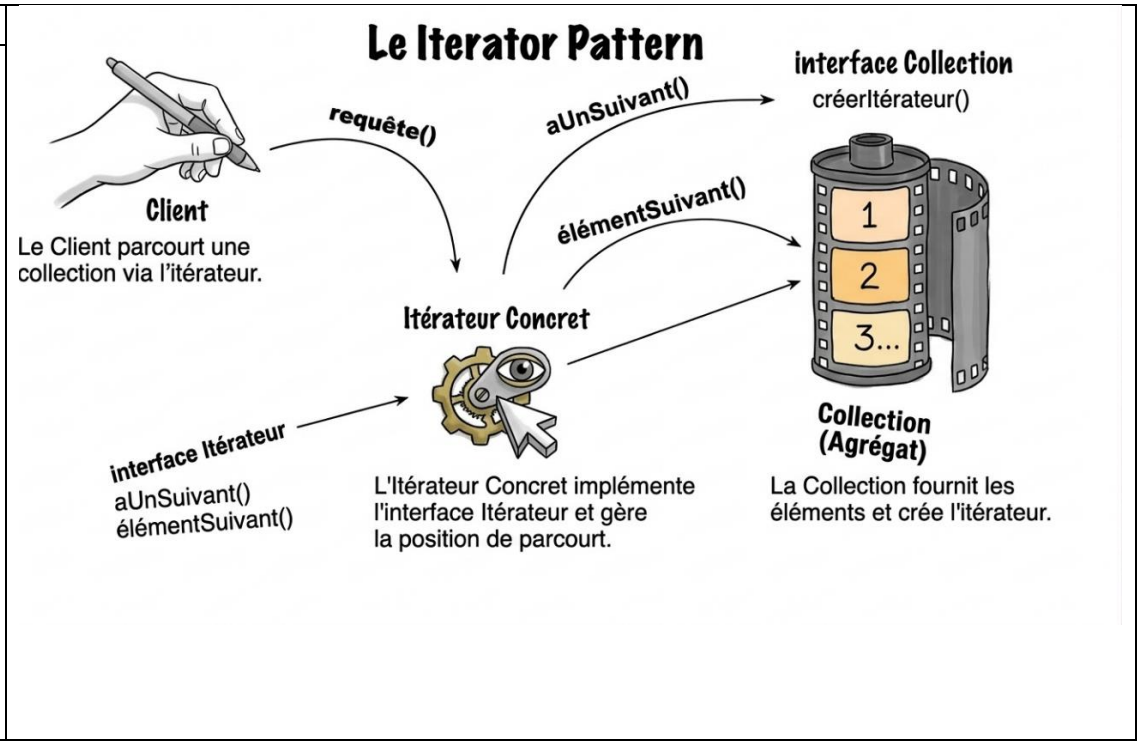
a. Les niveaux de cohesion: (**les exemples sont dans la class: ‘*org.springformation.solid.principles.examples.CohesionLevelsExamples***)

- Forte = Tâche complète + Objectif unique
- Parasite = Tâche complète + Objectifs mixtes
- Partielle = Tâche partielle + Objectif unique
- Faible = Tâche partielle + Objectifs mixtes

- Exemple : Le **Iterator Pattern : (voir les packages : *org.springformation.solid.principles.(propre/sale).iterator*)**
  - **La collection** a **une seule responsabilité** : **stocker et gérer les éléments**.
  - **L’itérateur** a **une seule responsabilité** : **gérer la logique de parcours (iteration)** des éléments.
    - **Note : L’itérateur masque également la structure de données interne, ce qui facilite son changement futur.**

Structure Standard





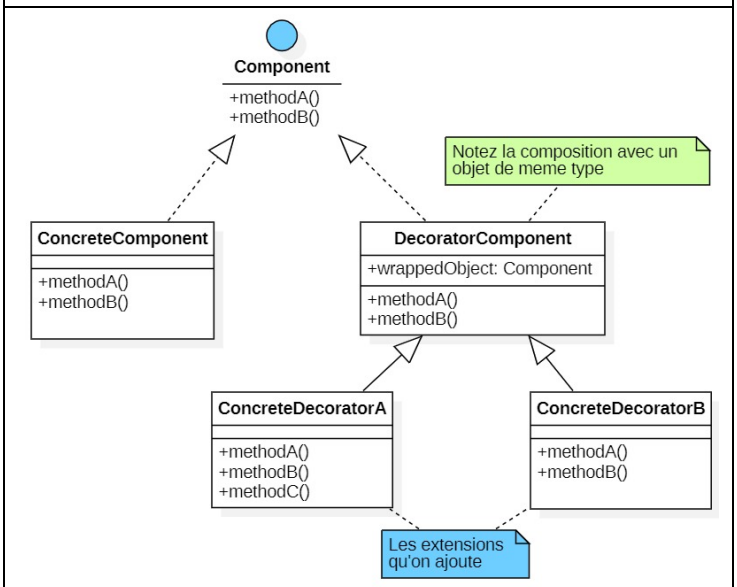
- **Note :** Java **intègre** déjà ce **design pattern à travers** l’interface ***java.util.Iterator***

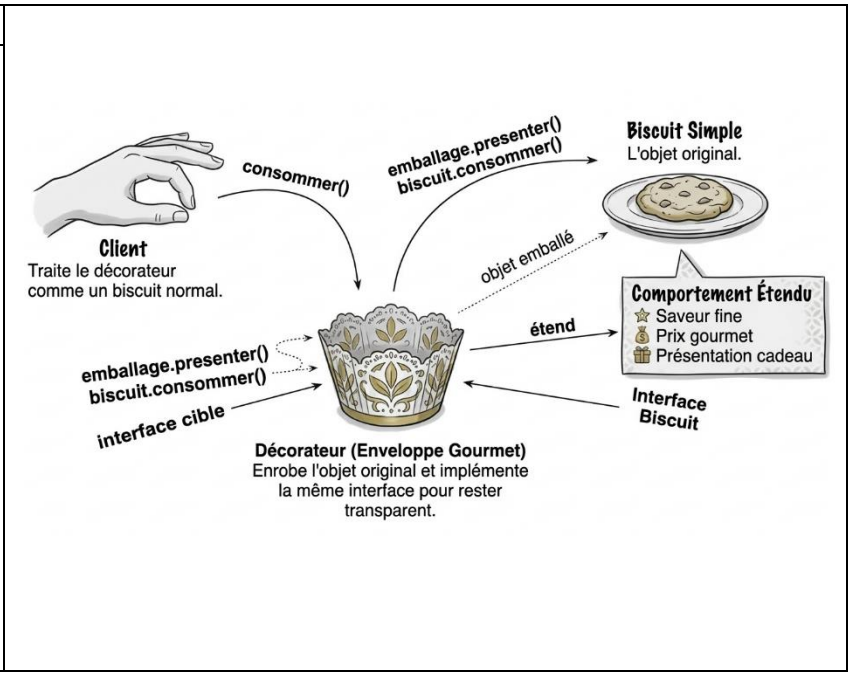
2. Open/Closed Principle :

Énoncé : **‘Une classe doit être ouverte à l'extension, mais fermée à la modification’**

- Le principe ouvert/fermé stipule qu’une classe doit pouvoir accueillir de nouvelles fonctionnalités par extension, sans que son code source d'origine ne soit altéré, ce qui garantit une évolution logicielle stable et sans risque de casser l'existant déjà testé.
- Exemple : Le **Decorator Pattern: (voir les packages: *org.springformation.solid.principles.(propre/sale).decorator*)**
  - **La classe de base est fermée à la modification : son code d'origine reste intact et n'a pas besoin d'être altéré.**
  - **Le décorateur permet l'extension : il ajoute de nouvelles fonctionnalités ou comportements au-dessus du composant de base, de manière dynamique.**

Structure Standard





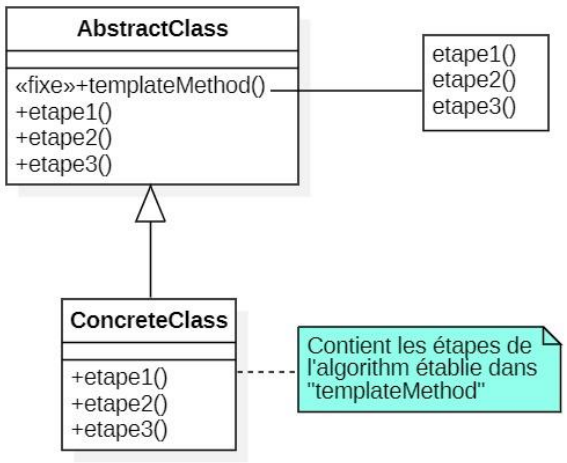
3. Liskov Substitution Principle :

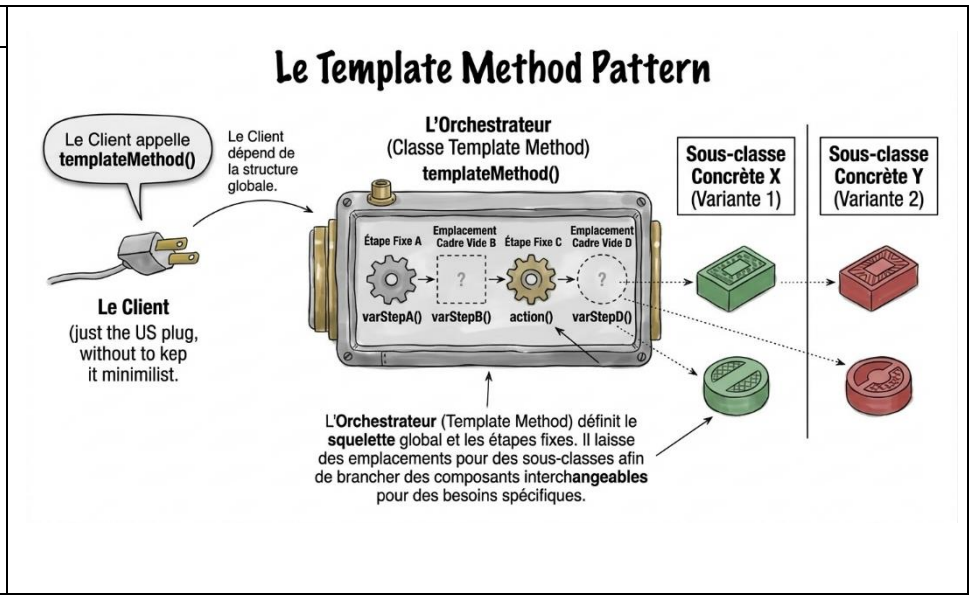
Énoncé : **‘Une classe dérivée doit pouvoir remplacer sa classe de base sans altérer le bon fonctionnement du programme’**

- Le principe de substitution de Liskov impose que n'importe quel objet d'une sous-classe puisse se substituer à un objet de sa classe parente sans briser la logique ou la stabilité du système, garantissant ainsi que les extensions respectent strictement le contrat établi par le type de base et ne provoquent aucun comportement inattendu lors de leur utilisation.

- Exemple : **Template Method Pattern**: (**voir les packages :** ***org.springformation.solid.principles.(propre/sale).template***)
    - **La classe de base définit la structure invariable de l'algorithme, garantissant que le comportement global et le contrat d'exécution restent fiables.**
    - **La classe dérivée implémente (substitue) les étapes spécifiques sans altérer la structure de l'algorithme initial, garantissant qu'elle peut remplacer la classe de base sans rien casser.**

Structure Standard

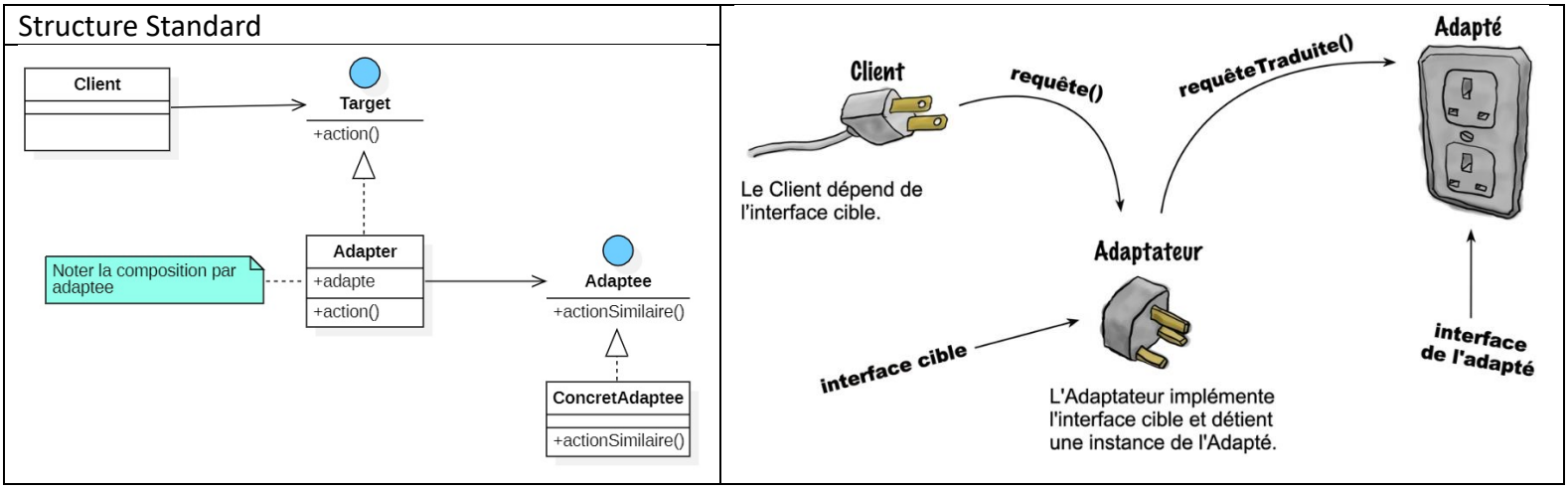




4. Interface Segregation Principle :

Énoncé : **‘Une classe ne devrait jamais être forcée de dépendre de méthodes ou d'interfaces qu'elle n'utilise pas’**

- Le principe de ségrégation des interfaces préconise de diviser les interfaces volumineuses en contrats plus petits et spécialisés, afin que chaque classe n'implémente que les fonctionnalités qui lui sont strictement utiles, évitant ainsi d'alourdir le code avec des méthodes superflues ou non pertinentes pour son rôle.
- Exemple : Le **Adapter Pattern**: (**voir les packages :** ***org.springformation.solid.principles.(propre/sale).adapter***)
    - **Le client a une seule interface cible à gérer, qui est petite, spécifique et taillée exactement pour ses besoins (il n'est pas encombré par des méthodes inutiles).**
    - **L'adaptateur se charge de faire le pont et d'encapsuler la classe tierce (qui possède souvent une interface trop large ou incompatible), afin d'exposer uniquement ce dont le client a besoin.**



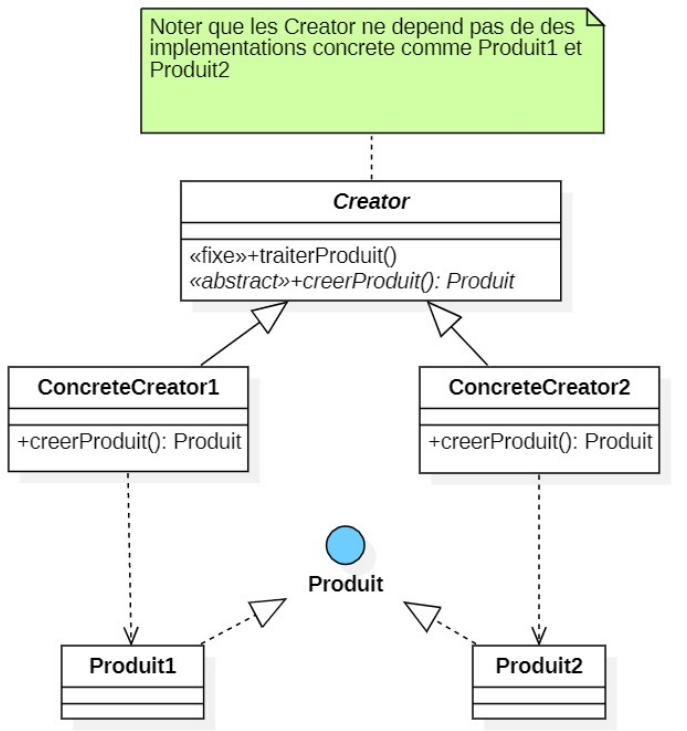
5. Dependency Inversion Principle :

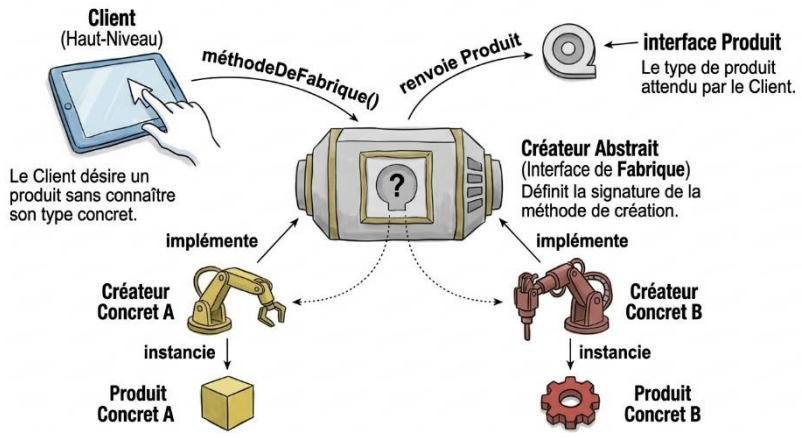
Énoncé : **‘Les classes de haut niveau ne doivent pas dépendre des classes de bas niveau ; les deux doivent dépendre d'abstractions et non d'implémentations concrètes’**

- Le principe d'inversion des dépendances stipule que les composants de haut niveau et de bas niveau doivent tous deux s'appuyer sur des abstractions plutôt que sur des implémentations concrètes, ce qui permet de découpler les différentes parties du système et de faciliter ainsi leur évolution ou leur remplacement sans impacter la logique métier globale.
- Des règles pour mieux appliquer ce principe :
  - **Aucune** variable ne doit contenir une **référence** vers une **classe concrète**.
  - **Aucune** classe ne doit **hériter** d’une **classe concrète**.
  - **Aucune** méthode ne doit **redéfinir** une **méthode déjà implémentée** dans l’une de ses classes de base.

- Exemple : Le Le **Factory Method Pattern**: **(voir les packages : *org.springformation.solid.principles.(propre/sale).factory*)**
  - Le code client (haut niveau) dépend uniquement d'une abstraction (l'interface du produit à créer), sans se soucier de la manière dont il est fabriqué.
  - La fabrique concrète (bas niveau) se charge de l'implémentation et de la création de l'objet, inversant ainsi la dépendance : le client ne fait plus de new ObjetConcret(), il dépend de l'abstraction.

Structure Standard





VI. Conclusion :

- L'objectif fondamental des bonnes pratiques, des principes **SOLID** et des **Design Patterns** n'est pas d'atteindre une perfection théorique, mais de préparer le logiciel à sa seule constante : **le changement**. En structurant le code autour d'abstractions plutôt que d'implémentations rigides, on facilite l'intégration de nouvelles fonctionnalités et on réduit drastiquement la dette technique.

- Cependant, dans la réalité du développement, il est virtuellement **impossible de satisfaire simultanément tous ces principes** sans risquer une "sur-ingénierie" (over-engineering) qui rendrait le système inutilement complexe. Le rôle crucial du développeur est alors de **définir ses priorités** : il doit évaluer la probabilité d'évolution d'un module. Si une partie du code est critique et sujette à des changements fréquents, l'application stricte de SOLID est un investissement vital. À l'inverse, pour un code stable et simple, une approche plus directe peut être préférable. En somme, la meilleure architecture est celle qui trouve le juste équilibre entre **flexibilité maximale** et **simplicité opérationnelle**.